

Codebase Review + Real Test-Run Report

1. Bug found and fixed

`evaluate_ablations.py` had two genuine Python syntax errors — not a copy/paste artifact, confirmed with `ast.parse()` :

```
logger.info("Main results table:
%s", main_table.to_string(index=False))  # unterminated string literal
```

(and the same pattern for "Per-severity table:"). This is an *unescaped literal newline* inside a single-quoted string, which is illegal Python and would raise `SyntaxError: unterminated string literal` the moment the module is imported — meaning **step 7 of `run_full_pipeline.py` (the paper figures/tables step) could never run**, and neither could `evaluate_ablations.py` be invoked standalone.

Fix: replaced with `"...\\n%s"` . Fixed copy attached: `evaluate_ablations_FIXED.py` — swap it in for the original before running the full pipeline.

All other 15 `.py` files parsed cleanly (`ast.parse` clean on every one) and I didn't find logic bugs that would crash a run — the design (fixed feature order, subject-level splitting, physiology-only labeling criterion, no-PSNR leakage into recipe selection, ffmpeg-based near-lossless intermediate encode to avoid `cv2.VideoWriter` contamination) is all internally consistent and does what the docstrings claim.

Environment check: Python 3.12, `ffmpeg` present, and every package in `requirements.txt` (`numpy/pandas/scipy/scikit-learn/opencv/scikit-image/joblib/matplotlib`) already installed at compatible versions. No install step was needed.

2. Does it auto-load MMPD/PURE, or are results synthetic?

No auto-download, and no synthetic substitution in the real pipeline.

- `dataset_loader.py`'s `load_ubfc_rppg / load_pure / load_mmpd` only **read files you already have on disk**, in specific expected folder layouts (e.g. MMPD needs `videos/ + labels.csv` with a `fitzpatrick` column). There's no network call and no fabrication — if the files aren't there, it raises `FileNotFoundError` or logs a skip warning, it does not invent numbers.
- The **only** synthetic-data generator anywhere in the codebase is `generate_synthetic_training_table()` in `router_classifier.py`, and it's quarantined behind the explicit `self-test CLI` subcommand with a hard-coded warning in the logs: *"produced PLACEHOLDER data... Do not report results trained on this in the paper."* It's not on any path that `run_full_pipeline.py` or the `train / predict` commands use.
- paper_ieee_draft.tex** **currently has no fabricated numbers in it** — every result is a bracketed placeholder (`[XX.X]` , `[N_UBFC]` , `[FF.F]` , etc.), and `README.md`'s example results table is explicitly flagged *"Replace with your actual experimental results after running the full pipeline."* So there's nothing in the current paper draft to un-hallucinate — good starting point.

Bottom line: to fill in the paper's bracketed placeholders for real, you need the actual UBFC-rPPG/PURE/MMPD files extracted and organized on disk, then run `dataset_loader.py load` → `degradation_injector.py sweep` → `labeling_harness.py run` → `router_classifier.py train` → `pipeline_runner.py batch` → `fairness_eval.py` → `evaluate_ablations.py` . None of that can be shortcut with 1 clip and no ground truth — see below for exactly what today's single-clip run *can* and *can't* tell you.

3. What I actually ran on `input_video.mp4`

The 69-subject UBFC `.crdownload` file isn't a usable dataset (it's a partial/incomplete browser download), so per your instruction I used the one finished `input_video.mp4` instead. Its real properties: **1920×1080, ~30 fps actual decode rate (261 frames / 8.73 s — note the container metadata says 60 fps but that's not what OpenCV actually decodes; more below), 8.7 s duration**. For the multi-recipe sweep I additionally made a 640×360 copy via `ffmpeg` purely to fit the compute budget you flagged — noted explicitly wherever it applies below.

All numbers below are **real outputs of the actual, unmodified project code** run just now — nothing here is fabricated or estimated by me.

3a. Degradation profile (raw clip, full 1080p, `degradation_estimator.py`)

Axis	Score (0–1)
noise	0.019
blur	0.362
compression	0.078
illumination	0.136
motion	0.060

Face detection rate: **100% (261/261 frames)**, both at 1080p and 640p.

3b. Pulse extraction / HR (no ground truth available for this clip)

This clip did not come with a synchronized contact-PPG/ECG signal, so **HR MAE cannot be computed** — only HR estimate, Pulse SNR, and SQI are meaningful here.

Recipe	HR (bpm)	Pulse SNR (dB)	SQI
R0 (raw)	112.5	19.72	0.972
R1 (denoise+CLAHE)	133.6	18.13	0.961
R2 (exposure+contrast)	182.8	19.47	0.983
R3 (white-balance+NLM)	154.7	15.90	0.880
R4 (CLAHE+unsharp)	140.6	16.61	0.875
R5 (gentle WB+bilateral+highboost)	119.5	21.65	0.989

(640×360, to fit compute — see caveat below)

With no ground truth, the labeling harness's own documented fallback applies: HR MAE is undefined for every recipe, so ranking collapses to its tiebreak rule (maximize Pulse SNR) → **winner = R5**.

Important honest caveat, not a code bug: the HR estimates above swing wildly (112–183 bpm) and are not physiologically plausible for a resting adult. Two likely real causes: (1) 8.7 s is far too short for a stable Welch PSD estimate — `nperseg=256` against a ~261-sample signal gives essentially one window and ~7 bpm frequency-bin resolution; UBFC/PURE/MMPD clips are all ≥30 s, which is what this pipeline is actually designed and calibrated for; (2) I also ran the raw clip at full 1080p and got HR=168.6 bpm vs 112.5 bpm at 640p for the *same* content — resolution sensitivity this large tells you the estimate isn't converged. **Don't put any of these bpm numbers in the paper** — they're a real diagnostic of pipeline mechanics working correctly, not a valid physiological measurement. You'd want a ≥30 s clip with known ground truth (i.e., an actual UBFC/PURE/MMPD subject) before trusting HR numbers from this pipeline.

3c. Track A synthetic-degradation round trip (real ffmpeg injection)

Generated real `noise_moderate` and `blur_moderate` degraded twins from the 640p clip via `degradation_injector.py`, then:

- `fidelity_metrics.py`: noise-degraded clip → PSNR=25.32 dB, SSIM=0.479 (real computed values, see `fidelity_noise_moderate.json`)
- `degradation_estimator.py validate`: no-reference estimator correctly ranked noise-score higher on the noise clip and blur-score higher on the blur clip (Pearson $r = +1.000$ on both axes) — but this is **only n=2 clips**, so that $r=1.000$ is a trivial 2-point correlation, not a real validation. The paper's actual validation figure needs the full grid (`degradation_injector.py sweep` across all severities) run on real clean source videos.

3d. `pipeline_runner.py single` (raw vs. fixed-best baselines)

Ran cleanly end-to-end. Adaptive baseline was skipped (as designed) because no router model exists yet — training one requires labeled clips with real ground-truth HR, which this single unlabeled clip can't provide.

4. What's still needed for real paper numbers

1. Get the actual UBFC-rPPG dataset fully downloaded (not `.crdownload`) and unpacked into the `subject/vid.avi + groundTruth.txt` layout `dataset_loader.py` expects; same for PURE and MMPD.
2. `dataset_loader.py load` → `manifest` → `degradation_injector.py sweep` → Track A grid → `labeling_harness.py run` (this is the compute-heavy step: 6 recipes × full rPPG pipeline × every clip) → `router_classifier.py train` → `pipeline_runner.py batch` → `fairness_eval.py` → `evaluate_ablations.py` (use the fixed copy).
3. Only then are the paper's `[XX.X]`, `[FF.F]`, `[N_UBFC]` etc. placeholders fillable with real numbers.

Attached: the fixed `evaluate_ablations.py`, and the raw JSON/CSV outputs from today's test run, for your own record-keeping.